

# KodiKit Operations Guide

---

Operating a self-updating Kodi add-on repository, and provisioning a Fire TV Stick 4K from it.

Two procedures. **Book One** sets up and runs the repository — the thing that keeps add-ons updating by themselves. **Book Two** takes a Fire TV Stick from factory state to a finished box in 34 numbered steps.

If the repository is already published, you only need Book Two.

Everything is written as steps to follow in order. Each step is one command to type or one control to click, and lines marked **Check** tell you what success looks like — if you do not see it, stop there rather than continuing. Background, file maps and troubleshooting live in the appendices so the procedures stay short.

# Contents

---

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>BOOK ONE</b>                                               | <b>3</b>  |
| Part 1 — First-time setup                                     | 4         |
| Part 2 — Publish to GitHub                                    | 5         |
| Part 3 — Add an add-on                                        | 6         |
| Part 4 — Remove an add-on                                     | 9         |
| Part 5 — Change the Toolbox itself                            | 9         |
| Part 6 — Change the repository's identity or URLs             | 9         |
| Part 7 — Daily operation                                      | 9         |
| Part 8 — Health check                                         | 11        |
| Appendix A — How it works                                     | 11        |
| Appendix B — File map                                         | 12        |
| Appendix C — Troubleshooting                                  | 12        |
| <b>BOOK TWO</b>                                               | <b>14</b> |
| Stage A — Prepare the stick (steps 1–5)                       | 16        |
| Stage B — Install Kodi (steps 6–8)                            | 16        |
| Stage C — Set up Surfshark (steps 9–13)                       | 16        |
| Stage D — Install the repository (steps 14–20)                | 18        |
| Stage E — Install the add-ons (steps 21–23)                   | 18        |
| Stage F — Configure (steps 24–30)                             | 19        |
| Stage G — Verify (steps 31–34)                                | 20        |
| After setup — routine maintenance                             | 20        |
| Troubleshooting                                               | 20        |
| Appendix A — The hardware                                     | 22        |
| Appendix B — What step 24 writes                              | 22        |
| Appendix C — VPN Manager with Surfshark, on a non-Android box | 23        |
| Appendix D — Why there is no snapshot build                   | 23        |

# BOOK ONE

---

Setting up and running the repository

Every section here is a numbered procedure. Run the steps in order; each one is a command to type or a control to click. Lines marked **Check** tell you what success looks like — if you do not see it, stop there rather than continuing.

Background and troubleshooting live in the appendices, so the procedures stay short.

## Part 1 — First-time setup

---

Do this once, on the machine where you will maintain the repository.

### 1.1 Verify your tools.

```
python3 --version    # need 3.8 or newer
git --version
gh --version         # optional, only for Part 2
```

### 1.2 Open the repository directory.

```
cd ~/Dev/kodi-repo
```

**1.3 Set your identity.** Open `repo.config.json` and set these four fields:

| Field                    | Set it to                                                  |
|--------------------------|------------------------------------------------------------|
| <code>github_user</code> | Your GitHub username or org, exactly as it appears in URLs |
| <code>github_repo</code> | The repository name you will push to                       |
| <code>repo_name</code>   | The display name shown in Kodi. Anything readable.         |
| <code>provider</code>    | Your name, shown as the author                             |

Leave `repo_id`, `branch`, `hosting` and `keep_versions` alone unless Part 6 applies.

### 1.4 Build.

```
python3 scripts/build_repo.py
```

**Check:** the last two lines report the add-on count, an md5, and the URL it will serve from. That URL must contain your `github_user` and `github_repo` from step 1.3.

### 1.5 Check dependencies.

```
python3 scripts/check_deps.py
```

**Check:** all dependencies resolve. If not, go to Appendix C.

### 1.6 Commit.

```
git add -A && git commit -m "Configure repository identity"
```

## Part 2 — Publish to GitHub

---

Do this once. After it, everything is automatic.

### 2.1 Sign in to GitHub.

```
gh auth login
```

**2.2 Create the repository.** It must be public — Kodi cannot authenticate.

```
gh repo create YOURNAME/YOURREPO --public --source=. --remote=origin
```

### 2.3 Push.

```
git push -u origin main
```

### 2.4 Wait for the build.

```
gh run watch
```

**Check:** the run completes green. If it fails, read the log — it is running the same two commands you ran in 1.4 and 1.5.

**2.5 Verify what Kodi will see.** Three URLs must respond correctly.

```
BASE=https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
curl -sI $BASE/addons.xml | head -1 # expect: HTTP/2 200
curl -s $BASE/addons.xml.md5 # expect: 32 hex characters, nothing else
curl -sI $BASE/repository.kodikit.zip | head -1 # expect: HTTP/2 200
```

**Check:** all three as described. A 404 on the first two means step 1.3 does not match where you actually pushed.

**2.6 Write down the install URL.** This is what you type into Kodi in Book Two:

```
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
```

## Part 3 — Add an add-on

The repeatable procedure. Six steps.

**3.1 Find the add-on's ID.** Not its display name. Any one of:

| Source            | Where to look                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Its GitHub repo   | <code>addon.xml</code> → the <code>id=</code> attribute of the root <code>&lt;addon&gt;</code> element |
| An installed copy | The folder name under <code>~/.kodi/addons/</code> , e.g. <code>script.trakt/</code>                   |
| Kodi              | Add-ons → the add-on → Information                                                                     |

**3.2 Decide which section it goes in.** Run this, with your IDs in the `probe` list:

```
python3 - <<'EOF'
import gzip, re, urllib.request
url = "https://mirrors.kodi.tv/addons/omega/addons.xml.gz"
req = urllib.request.Request(url, headers={"User-Agent": "kodikit"})
raw = gzip.decompress(urllib.request.urlopen(req, timeout=60).read()).decode("utf-8",
"replace")
ids = set(re.findall(r'<addon\s+id="([^\"]+)"', raw))
for probe in ["script.trakt", "service.vpn.manager"]:
    print("%-34s %s" % (probe, "IN official" if probe in ids else "NOT in official"))
EOF
```

| Result                                 | Section  | Why                                                       |
|----------------------------------------|----------|-----------------------------------------------------------|
| IN official                            | official | Installed by ID; stays on the maintainer's update channel |
| NOT in official                        | mirror   | Packaged into your repo from its GitHub source            |
| Installs only from a vendor's own repo | external | Documented, never auto-installed — see Appendix A         |

**3.3 Add the entry to `addons.json`.** Copy the matching shape.

For `mirror`:

```
{
  "id": "service.vpn.manager",
  "name": "VPN Manager for OpenVPN",
  "category": "privacy",
  "why": "One sentence on what it does and why it earned a slot.",
  "github": "Zomboided/service.vpn.manager",
  "track": "release",
  "optional": true
}
```

For `official` — no `github`, no `track`:

```
{
  "id": "script.trakt",
  "name": "Trakt",
  "category": "tracking",
  "why": "Scrobbling, watched-state sync and lists across devices."
}
```

For `external` — different shape entirely: no `id`, a `vendor url`, and the add-ons it provides:

```
{
  "name": "TheMovieDb Helper (6.x)",
  "why": "Hard-requires script.module.pil, which no official repo ships.",
  "url": "https://jurialmunkey.github.io/repository.jurialmunkey/",
  "addons": ["plugin.video.themoviedb.helper", "script.skinvariables"]
}
```

Field reference:

| Field                 | Sections         | Meaning                                                                                        |
|-----------------------|------------------|------------------------------------------------------------------------------------------------|
| <code>id</code>       | mirror, official | Must match the add-on's own <code>addon.xml</code> exactly                                     |
| <code>name</code>     | all              | Display name in the Toolbox picker                                                             |
| <code>category</code> | mirror, official | Groups and sorts the picker. In use: subtitles, playback, tracking, tools, interface, privacy. |
| <code>why</code>      | all              | Your justification. Ends up in the README.                                                     |
| <code>github</code>   | mirror           | owner/repo                                                                                     |
| <code>track</code>    | mirror           | release (newest release, falling back to branch head) or branch                                |
| <code>ref</code>      | mirror           | Pin a branch, e.g. <code>"ref": "matrix"</code>                                                |
| <code>optional</code> | mirror, official | <code>true</code> leaves it <b>unticked</b> in the picker                                      |

**3.4 Bump the Toolbox version.** Open `src/script.kodikit.toolbox/addon.xml` and increase the patch number, e.g. `1.2.1` → `1.2.2`.

**Do not skip this.** The curated list ships *inside* the Toolbox. Change the list without changing the version and the published index stays byte-identical, so no device ever fetches your new list — while the build looks completely successful. `build_repo.py` now refuses to finish in that state and names the stale add-on.

### 3.5 Build and check.

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py
```

**Check:** exit code 0, and all dependencies resolve. If the build stops with `content changed without a version bump`, you missed 3.4.

### 3.6 Push.

```
git add -A && git commit -m "Add <addon-id>" && git push
```

**Check:** run the verification in Part 8.



## Part 4 — Remove an add-on

---

**4.1** Delete its entry from `addons.json`.

**4.2** Bump the Toolbox version in `src/script.kodikit.toolbox/addon.xml`.

**4.3** Build and push:

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py  
git add -A && git commit -m "Remove <addon-id>" && git push
```

**Check:** `grep '<addon id=<addon-id>'' docs/addons.xml` returns nothing.

Copies already installed on devices keep working but stop receiving updates. A repository cannot retract an add-on — to remove it from a device, uninstall it there.

## Part 5 — Change the Toolbox itself

---

**5.1** Edit the code under `src/script.kodikit.toolbox/`.

**5.2** Bump the version in its `addon.xml` — patch for a fix, minor for new behaviour.

**5.3** Build, check, push:

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py  
git add -A && git commit -m "Toolbox: <what changed>" && git push
```

## Part 6 — Change the repository's identity or URLs

---

Only when moving the repo or renaming it.

**6.1** Edit `repo.config.json`: `github_user`, `github_repo`, `branch`, or `hosting`.

**6.2** Bump `repo_version` in the same file.

**6.3** Build and push as in 5.3.

**6.4** Reinstall the repository zip on **every device**. The old URLs are baked into the copy they already have.

Because of 6.4, get `github_user` and `github_repo` right in Part 1 and leave them alone.

## Part 7 — Daily operation

---

Nothing to do. For reference, the scheduled job at 05:15 UTC:

1. Checks each mirrored add-on's upstream for a newer release.
2. Repackages anything that moved.
3. Runs the dependency check; a failure stops the commit.
4. Commits the regenerated docs/ only if something changed.

Devices poll roughly every 24 hours, so worst case is about 48 hours from an upstream release to your stick.

To force a run now:

```
gh workflow run sync.yml && gh run watch
```

## Part 8 — Health check

---

Run after any push, or whenever something looks wrong.

### 8.1 The index lists what you expect.

```
grep -o 'id="[^\"]*" name="[^\"]*" version="[^\"]*" ' docs/addons.xml
```

### 8.2 A specific add-on is present.

```
grep -o 'id="service.vpn.manager"[^>]*version="[^\"]*" ' docs/addons.xml
```

### 8.3 Its zip is Kodi-shaped — `<id>/addon.xml` must be at the root:

```
unzip -l docs/zips/service.vpn.manager/*.zip | head -5
```

### 8.4 The Toolbox carries the current list.

```
python3 -c "import zipfile,json,glob; z=sorted(glob.glob('docs/zips/script.kodikit.toolbox/*.zip'))[-1]; \
m=json.loads(zipfile.ZipFile(z).read('script.kodikit.toolbox/resources/addons.json')); \
print(sorted(e['id'] for s in ('mirror','official') for e in m[s]))"
```

### 8.5 The live URLs still respond. Re-run 2.5.

**8.6 On a device:** Toolbox → Install curated add-ons. New entries appear in the picker, unticked if you marked them `optional`.

## Appendix A — How it works

---

**The update loop.** Kodi downloads `addons.xml.md5` (32 bytes) on a ~24 hour cycle. If that hash changed, it downloads `addons.xml`, compares every `version=` against what is installed, and downloads anything newer from `docs/zips/`. So an update happens **only** when a version number increases. Re-uploading a zip at the same version does nothing; lowering a version does nothing. A broken release is fixed with a higher patch version, never a re-upload.

**The three routes.** `official` keeps an add-on on its maintainer's update channel — mirroring something Kodi already ships would fork it onto your channel and make you responsible for tracking it. `mirror` is for add-ons genuinely unavailable elsewhere. `external` is for add-ons whose dependencies nobody publishes: TheMovieDb Helper 6.x hard-requires `script.module.pil`, a compiled Pillow build no official repo ships, so mirroring it would deliver an add-on that cannot install.

**What the builder does.** Syncs mirrors from upstream; packages everything in `src/` at the version in its `addon.xml`; generates the repository add-on from `repo.config.json`; regenerates the index, checksum, `index.html`, and prunes old zips. Zips use fixed timestamps, so identical content produces identical bytes — otherwise CI would commit noise daily.

**What the dependency checker does.** Fetches Kodi's official index for Omega and Piers, unions it with your own IDs, and verifies every non-optional `<import>` resolves. It fails the build rather than letting a device discover the problem.

**Where the add-on ID appears.** Folder name in `docs/zips/`, zip filename, folder at the root inside the zip, and the `id=` in `addon.xml`. Kodi builds its download URL from the ID and version in the index, so any mismatch is a 404 that surfaces as "failed to install". The builder derives all four from one source, which is why you never rename zips by hand.

## Appendix B — File map

| Path                               | Purpose                                     | Edit?                                    |
|------------------------------------|---------------------------------------------|------------------------------------------|
| <code>addons.json</code>           | The curated list                            | <b>Yes — this is the control surface</b> |
| <code>repo.config.json</code>      | Repo identity, hosting, versions            | Yes                                      |
| <code>src/</code>                  | Add-ons you maintain                        | Yes                                      |
| <code>scripts/build_repo.py</code> | The builder                                 | Rarely                                   |
| <code>scripts/check_deps.py</code> | Dependency validation                       | Rarely                                   |
| <code>scripts/make_art.py</code>   | Regenerates icon/fanart (needs Pillow)      | Rarely                                   |
| <code>assets/</code>               | Committed icon and fanart                   | Rarely                                   |
| <code>docs/</code>                 | <b>Published tree — what Kodi downloads</b> | <b>Never by hand</b>                     |
| <code>guides/</code>               | This document's source                      | Yes                                      |

## Appendix C — Troubleshooting

| Symptom                                                        | Cause                          | Fix                                                                    |
|----------------------------------------------------------------|--------------------------------|------------------------------------------------------------------------|
| Build stops: content changed without a version bump            | The working case, caught       | Bump the version in <code>src/&lt;addon&gt;/addon.xml</code> , rebuild |
| <code>check_deps</code> reports an unresolvable import         | A dependency nobody publishes  | Move the add-on to external and document the vendor repo               |
| Mirror sync: no published release, falling back to branch head | Upstream publishes no releases | Normal. Set <code>"track": "branch"</code> to make it explicit.        |

| Symptom                                      | Cause                                                            | Fix                                    |
|----------------------------------------------|------------------------------------------------------------------|----------------------------------------|
| Mirror sync: add-on not found in the tarball | <code>id</code> does not match upstream's <code>addon.xml</code> | Correct the <code>id</code>            |
| Mirror sync fails after an upstream rename   | Default branch changed                                           | Set "ref" explicitly                   |
| Kodi: "Could not connect to repository"      | <code>github_user/github_repo</code> wrong, or repo private      | Re-run 2.5; confirm the repo is public |
| Kodi: repository installs but lists nothing  | Index empty, or checksum stale                                   | Re-run 1.4; never hand-edit docs/      |
| Kodi: "Failed to install add-on"             | Zip name does not match <code>id-version.zip</code>              | Re-run 1.4; the builder names them     |
| Kodi: "Dependency not met"                   | Unresolvable <code>&lt;import&gt;</code>                         | Run 1.5                                |
| An add-on never updates on one device        | Per-add-on auto-update set to Never                              | Add-on info → Auto-update → On         |
| Updates appear only after a restart          | Normal 24-hour poll cycle                                        | Force a check on the device, or wait   |

# BOOK TWO

---

Provisioning a Fire TV Stick, step by step

One continuous procedure, steps 1 to 34, taking a Fire TV Stick from factory state to a finished, self-updating Kodi box. Target hardware: **Fire TV Stick 4K (2018, model E9L29Y)** — 1.5 GB RAM, 8 GB storage, Android 7.1+, Kodi 21.1. Notes for the 1 GB gen 1/2 sticks appear where they differ.

Allow about 30 minutes. Lines marked **Check** tell you what success looks like.

**Before you start, have these ready:**

- The install URL from Book One, step 2.6
- Your Surfshark login
- The Kodi `armeabi-v7a` APK for your version, or the Downloader app
- Optional but recommended: ADB on your computer, for the verification steps

## Stage A — Prepare the stick (steps 1–5)

---

1. On the stick: Settings → My Fire TV → About → click **Fire TV Stick** seven times to reveal Developer Options.
2. Settings → My Fire TV → Developer Options → **Apps from Unknown Sources** → **On**.
3. (Optional, for the verification steps) Same menu → **ADB debugging** → **On**.
4. Settings → My Fire TV → About → Network → note the **IP address**.
5. (Optional) From your computer:

```
adb connect STICK-IP:5555
```

**Check:** connected to STICK-IP:5555. Accept the prompt on the TV if one appears.

## Stage B — Install Kodi (steps 6–8)

---

6. Install Kodi, by either route:

*Downloader app:* open Downloader on the stick, enter the Kodi download URL for the **armeabi-v7a** build, install when it downloads.

*ADB from your computer:*

```
adb install -r kodi-21.1-0mega-armeabi-v7a.apk
```

Use **armeabi-v7a**, not arm64-v8a. The stick is 32-bit ARM; the wrong APK simply fails to install.

7. Launch Kodi once, let it reach the home screen, then quit it. This creates the profile directory.
8. Skip Kodi's setup wizard if offered. Leave the skin as Estuary.

## Stage C — Set up Surfshark (steps 9–13)

---

Do this **before** any sign-in inside Kodi. Sign-ins bind to the address that completed them, so doing this later means redoing them.

9. On the stick: Amazon appstore → search **Surfshark** → install.
10. Open Surfshark, sign in, connect to a location.
11. In Surfshark's settings, enable **auto-connect on startup**, so a reboot cannot silently drop protection.
12. (Optional) Set **Bypasser** if you want specific apps outside the tunnel.
13. Verify the tunnel at OS level — the app's own UI is not proof:



```
adb shell ip addr show tun0
```

**Check:** a `tun0` interface exists with an IP. If it does not, the stick is not tunnelling no matter what the app displays.

**Do not use the VPN Manager add-on on this device.** It raises an OpenVPN tunnel from inside Kodi, which an ordinary Android app cannot do on Fire OS. It also bundles profiles for NordVPN, ExpressVPN and PIA only — not Surfshark. It stays in the repository, unticked, for Linux and LibreELEC boxes; Appendix C covers using it with Surfshark there.

## Stage D — Install the repository (steps 14–20)

14. Kodi → Settings (gear) → System → Add-ons → **Unknown sources** → **On**. Accept the warning.

15. Settings → Add-ons → Updates → **Install updates automatically**.

Step 15 goes before any add-on exists so every one inherits it. This single setting is what makes the stick maintenance-free.

16. Settings → File manager → **Add source** → `<None>`.

17. Type the install URL from Book One, step 2.6:

```
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
```

18. Name it `kodikit` → OK.

19. Kodi → Add-ons → **Install from zip file** → `kodikit` → `repository.kodikit.zip`.

**Check:** a notification confirms the repository add-on installed.

20. Add-ons → **Install from repository** → KodiKit → Program add-ons → **KodiKit Toolbox** → Install.

**Check:** the Toolbox appears under Add-ons → Program add-ons.

If step 19 fails, the URL in step 17 is wrong. Re-run Book One step 2.5 from your computer.

## Stage E — Install the add-ons (steps 21–23)

21. Open **KodiKit Toolbox** → **Install curated add-ons**.

22. In the picker, untick anything you do not want. For this stick:

| Untick                              | Reason                                              |
|-------------------------------------|-----------------------------------------------------|
| <code>skin.copacetic</code>         | 40–150 MB and slower menus; Estuary is lighter      |
| <code>script.plexmod</code>         | Only useful with a Plex server                      |
| <code>pvr.iptvsimple</code>         | Only useful with an IPTV M3U subscription           |
| <code>service.vpn.manager</code>    | Wrong route on Fire OS — Stage C already covered it |
| <code>script.service.janitor</code> | Only if the stick stores local recordings           |

Leave ticked: InputStream Adaptive and FFmpeg Direct, a4kSubtitles, Black Bars Never, Up Next, Logfile Uploader, YouTube.

23. Confirm and let it run.

**Check:** the Toolbox reports what installed. Anything already present is skipped automatically.

## Stage F — Configure (steps 24–30)

---

**24.** Toolbox → **Apply streaming cache tuning** → select **Fire TV Stick** → Write → **Restart** when prompted.

Do not pick *Balanced* here. Its 64 MB buffer becomes roughly 192 MB resident, which does not fit alongside Fire OS on 1.5 GB. On a 1 GB gen 1/2 stick, pick *Low memory* instead.

**25.** Settings → Player → Language → **Default TV show service** and **Default movie service** → set both to **a4kSubtitles**.

Subtitles do nothing until this is set. It is the single most common "it's broken" report.

**26.** Settings → Player → Videos → **Allow hardware acceleration – MediaCodec (Surface)** → **On**.

**27.** Same screen → **Allow hardware acceleration – MediaCodec** → **Off** (redundant with Surface, costs memory).

**28.** Same screen → **Adjust display refresh rate** → **On start/stop**; **Sync playback to display** → **Off**; **Enable HQ scalers** → **0%**.

**29.** Settings → Interface → **Show RSS news feeds** → **Off**. Settings → Media → General → **Show media flags** → **Off**.

**30.** Settings → Services → Control → **Allow remote control via HTTP** → **On**, port 8080.

Step 30 is what makes remote diagnosis possible later without touching the device.

## Stage G — Verify (steps 31–34)

**31.** Reboot the stick, then leave Kodi sitting on the home screen for a minute.

**32.** Check memory and disk from your computer:

```
adb shell dumpsys meminfo org.xbmc.kodi | grep 'TOTAL PSS'
adb shell du -sh /sdcard/Android/data/org.xbmc.kodi/files/.kodi
```

**Check:** against the targets below.

| Metric              | Stick 4K     | 1 GB sticks  |
|---------------------|--------------|--------------|
| TOTAL PSS idle      | under 600 MB | under 420 MB |
| .kodi/ on disk      | under 500 MB | under 350 MB |
| Cold start          | under 18 s   | under 25 s   |
| Background services | 6 or fewer   | 4 or fewer   |

**33.** Confirm auto-update really is set:

```
adb shell "grep addonupdates \
/sdcard/Android/data/org.xbmc.kodi/files/.kodi/userdata/guisettings.xml"
```

**Check:** the value is 0. If it is 1 or 2, redo step 15.

**34.** Play something for a minute. Press **Ctrl+Shift+0** (or hold Select on the remote → the codec overlay) and confirm no dropped frames.

**Done.** The stick now updates itself. Every add-on tracks its own upstream through the repository, with no further action.

## After setup — routine maintenance

| When                   | Do                                                 |
|------------------------|----------------------------------------------------|
| Monthly                | Toolbox → <b>Storage report</b>                    |
| When storage looks bad | Toolbox → <b>Clean up caches</b>                   |
| Occasionally           | Toolbox → <b>Clean video/music library</b>         |
| After adding an add-on | Re-run step 32 to catch a heavy background service |

Artwork re-downloads as you browse after a cache clean. That is expected; the first few minutes feel slower.

## Troubleshooting

| Symptom                                                | Fix                                                                    |
|--------------------------------------------------------|------------------------------------------------------------------------|
| Kodi APK will not install                              | Wrong architecture — use <code>armeabi-v7a</code>                      |
| Step 19 fails                                          | URL in step 17 is wrong; re-run Book One step 2.5                      |
| Repository installs but lists nothing                  | Book One, Appendix C                                                   |
| Subtitles never appear                                 | Step 25 not done                                                       |
| Buffering on 1080p or 4K                               | Redo step 24 and pick <b>Fire TV Stick</b>                             |
| Stutter on high-bitrate video                          | Step 26 — MediaCodec (Surface) is off                                  |
| Black screen with audio                                | Step 27 — disable the non-Surface MediaCodec option                    |
| Very slow menus                                        | A third-party skin got installed; switch back to Estuary               |
| Out of space within weeks                              | Toolbox → Clean up caches; confirm step 24 was applied                 |
| Add-ons never update                                   | Step 33 returns something other than 0                                 |
| Surfshark shows connected but traffic is not tunnelled | Step 13 shows no <code>tun0</code> ; reconnect in the app              |
| Everything is slow on a gen 1/2 stick                  | 1 GHz dual-core is the limit — cut the add-on count, expect 1080p only |

## Appendix A — The hardware

|                        | Stick gen 1 (2014) | Stick gen 2 (2016) | Stick 4K (2018)          | Stick Lite / 3rd gen |
|------------------------|--------------------|--------------------|--------------------------|----------------------|
| Model                  | W87CUN             | LY73PR             | <b>E9L29Y</b>            | —                    |
| RAM                    | 1 GB               | 1 GB               | <b>1.5 GB</b>            | 1 GB                 |
| Storage                | 8 GB (~5 usable)   | 8 GB (~5 usable)   | <b>8 GB (~5 usable)</b>  | 8 GB                 |
| CPU                    | Dual-core 1.0 GHz  | Quad-core 1.3 GHz  | <b>Quad-core 1.7 GHz</b> | Quad-core 1.7 GHz    |
| Android base           | 5.1                | 5.1                | <b>7.1+</b>              | 9                    |
| Realistic Kodi ceiling | 19–20              | 20                 | <b>21+</b>               | 21+                  |

Fire OS reserves 350–450 MB. Kodi with Estuary is 250–300 MB. That leaves roughly 250 MB of headroom on a 1 GB stick and 700 MB on the 4K.

**Storage, not RAM, is the binding constraint on the 4K.** The extra 512 MB buys room for services, but the flash is the same 8 GB and the texture cache grows without limit until something caps it — which is what step 24 does.

On gen 1/2 sticks (Android 5.1) the practical Kodi ceiling is 20; check [kodi.tv/download](http://kodi.tv/download) for the current `armeabi-v7a` minimum, as it rises with each release.

## Appendix B — What step 24 writes

Toolbox → cache tuning → Fire TV Stick produces `userdata/advancedsettings.xml`, backing up any existing file with a timestamp first:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<advancedsettings>
  <cache>
    <buffermode>1</buffermode>
    <memorysize>41943040</memorysize>
    <readfactor>4</readfactor>
  </cache>
  <gui>
    <algorithmdirtyregions>3</algorithmdirtyregions>
  </gui>
  <imageres>540</imageres>
  <fanartres>720</fanartres>
  <network>
    <curlclienttimeout>20</curlclienttimeout>
    <curlallowspendtime>15</curlallowspendtime>
  </network>
</advancedsettings>
```

- `memorysize` **40 MB**. Kodi allocates roughly three times the configured value, so this is ~120 MB resident. *Low memory* (20 MB → ~60 MB) suits 1 GB sticks; *Balanced* (64 MB → ~192 MB) needs a box with real RAM.
- `imageres / fanartres`. Dropping fanart from 1080 to 720 more than halves texture cache growth, and on a TV that upscales anyway the difference is invisible. This is the most effective single setting against the 8 GB flash filling.
- `algorithmdirtyregions` **3**. Redraw only what changed.
- **Network timeouts**. A dead source fails instead of hanging the UI.

Cache settings take effect only after the restart in step 24.

## Appendix C — VPN Manager with Surfshark, on a non-Android box

---

Only relevant on Linux, LibreELEC or Windows, where an add-on can execute OpenVPN.

1. Toolbox → Install curated add-ons → tick **VPN Manager for OpenVPN**.
2. VPN Manager → settings → provider → **User Defined**.
3. Download Surfshark's `.ovpn` config files from their manual-setup page.
4. Import them with VPN Manager's import wizard.
5. Authenticate with Surfshark's **manual-setup service credentials** from your account dashboard — a separate username and password, **not** your Surfshark login.

Step 5 is the usual failure: the account email and password do not work for manual OpenVPN.

## Appendix D — Why there is no snapshot build

---

The original goal was a "build" — a packaged `userdata` + `addons` snapshot installed by a wizard. That shape is wrong here for four reasons:

1. **It cannot update itself**. Add-ons inside a build are sideloaded, belong to no repository, and never receive updates.
2. **Reinstalling wipes your configuration**, because updating means replacing `userdata`.
3. **Builds rot across Kodi releases** — settings move and the add-on database schema changes.
4. **Builds leak credentials**. `userdata/addon_data/` holds API keys and account logins; packaging it ships your accounts to everyone who installs it.

Stages D–F deliver what people actually want from a build — one action, whole set installed, sensible settings — while every add-on stays individually updatable.

**If you genuinely need one** (cloning many identical sticks, or no reliable network):

```
cd ~/.kodi
zip -qr ~/mybuild-1.0.0.zip addons userdata \
  -x 'addons/packages/*' 'userdata/Database/*' 'userdata/Thumbnails/*' \
```

```
'*/__pycache__/*' '*.pyc' '*.log' 'temp/*'  
grep -rile 'token|apikey|password|secret' ~/.kodi/userdata/addon_data/
```

Review whatever that `grep` lists before sharing the zip. Restore by extracting over `.kodi/` with Kodi **force-stopped** — it rewrites `guisettings.xml` on exit and will otherwise overwrite what you restored:

```
adb shell am force-stop org.xbmc.kodi  
adb push mybuild-1.0.0.zip /sdcard/Download/  
adb shell "cd /sdcard/Android/data/org.xbmc.kodi/files/.kodi && \  
unzip -o /sdcard/Download/mybuild-1.0.0.zip"
```

Install the repository afterwards so the cloned devices resume receiving updates.